Chapter 8. Managing Volunteers

Getting people to agree on what a project needs, and to work together to achieve it, requires more than just a genial atmosphere and a lack of obvious dysfunction. It requires someone, or several someones, consciously managing all the people involved. Managing volunteers may not be a technical craft in the same sense as computer programming, but it is a craft in the sense that it can be improved through study and practice.

This chapter is a grab-bag of specific techniques for managing volunteers. It draws, perhaps more heavily than previous chapters, on the Subversion project as a case study, partly because I was working on that project as I wrote this and had all the primary sources close at hand, and partly because it's more acceptable to cast critical stones into one's own glass house than into others'. But I have also seen in various other projects the benefits of applying—and the consequences of not applying—the recommendations that follow; when it is politically feasible to give examples from some of those other projects, I will do so.

Speaking of politics, this is as good a time as any to drag that much-maligned word out for a closer look. Many engineers like to think of politics as something other people engage in. "*I'm* just advocating the best course for the project, but *she's* raising objections for political reasons." I believe this distaste for politics (or for what is imagined to be politics) is especially strong in engineers because engineers are bought into the idea that some solutions are objectively superior to others. Thus, when someone acts in a way that seems motivated by outside considerations—say, the maintenance of his own position of influence, the lessening of someone else's influence, outright horse-trading, or avoiding hurting someone's feelings—other participants in the project may get annoyed. Of course, this rarely prevents them from behaving in the same way when their own vital interests are at stake.

If you consider "politics" a dirty word, and hope to keep your project free of it, give up right now. Politics are inevitable whenever people have to cooperatively manage a shared resource. It is absolutely rational that one of the considerations going into each person's decision-making process is the question of how a given action might affect his own future influence in the project. After all, if you trust your own judgement and skills, as most programmers do, then the potential loss of future influence has to be considered a technical result, in a sense. Similar reasoning applies to other behaviors that might seem, on their face, like "pure" politics. In fact, there is no such thing as pure politics: it is precisely because actions have multiple real-world consequences that people become politically conscious in the first place. Politics is, in the end, simply an acknowledgment that *all* consequences of decisions must be taken into account. If a particular decision leads to a result that most participants find technically satisfying, but involves a change in power relationships that leaves key people feeling isolated, the latter is just as important a result as the former. To ignore it would not be high-minded, but shortsighted.

So as you read the advice that follows, and as you work with your own project, remember that there is *no one* who is above politics. Appearing to be above politics is merely one particular political strategy, and sometimes a very useful one, but it is never the reality. Politics is simply what happens when people disagree, and successful projects are those that evolve political mechanisms for managing disagreement constructively.

Getting the Most Out of Volunteers

Why do volunteers work on free software projects?[1]

When asked, many claim they do it because they want to produce good software, or want to be personally involved in fixing the bugs that matter to them. But these reasons are usually not the whole story. After all, could you imagine a volunteer staying with a project even if no one ever said a word in appre-

[1]This question was studied in detail, with interesting results, in a paper by Karim Lakhani and Robert G. Wolf, entitled *Why Hackers Do What They Do: Understanding Motivation and Effort in Free/Open Source Software Projects* . See http://freesoftware.mit.edu/papers/lakhaniwolf.pdf.

ciation of his work, or listened to him in discussions? Of course not. Clearly, people spend time on free software for reasons beyond just an abstract desire to produce good code. Understanding volunteers' true motivations will help you arrange things so as to attract and keep them. The desire to produce good software may be among those motivations, along with the challenge and educational value of working on hard problems. But humans also have a built-in desire to work with other humans, and to give and earn respect through cooperative activities. Groups engaged in cooperative activities must evolve norms of behavior such that status is acquired and kept through actions that help the group's goals.

Those norms won't always arise by themselves. For example, on some projects—experienced open source developers can probably name several off the tops of their heads—people apparently feel that status is acquired by posting frequently and verbosely. They don't come to this conclusion accidentally; they come to it because they are rewarded with respect for making long, intricate arguments, whether or not that actually helps the project. Following are some techniques for creating an atmosphere in which status-acquiring actions are also constructive actions.

Delegation

Delegation is not merely a way to spread the workload around; it is also a political and social tool. Consider all the effects when you ask someone to do something. The most obvious effect is that, if he accepts, he does the task and you don't. But another effect is that he is made aware that you trusted him to handle the task. Furthermore, if you made the request in a public forum, then he knows that others in the group have been made aware of that trust too. He may also feel some pressure to accept, which means you must ask in a way that allows him to decline gracefully if he doesn't really want the job. If the task requires coordination with others in the project, you are effectively proposing that he become more involved, form bonds that might not otherwise have been formed, and perhaps become a source of authority in some subdomain of the project. The added involvement may be daunting, or it may lead him to become engaged in other ways as well, from an increased feeling of overall commitment.

Because of all these effects, it often makes sense to ask someone else to do something even when you know you could do it faster or better yourself. Of course, there is sometimes a strict economic efficiency argument for this anyway: perhaps the opportunity cost of doing it yourself would be too high—there might be something even more important you could do with that time. But even when the opportunity cost argument doesn't apply, you may *still* want to ask someone else to take on the task, because in the long run you want to draw that person deeper into the project, even if it means spending extra time watching over them at first. The converse technique also applies: if you occasionally volunteer for work that someone else doesn't want or have time to do, you will gain his good will and respect. Delegation and substitution are not just about getting individual tasks done; they're also about drawing people into a closer committment to the project.

Distinguish clearly between inquiry and assignment

Sometimes it is fair to expect that a person will accept a particular task. For example, if someone writes a bug into the code, or commits code that fails to comply with project guidelines in some obvious way, then it is enough to point out the problem and thereafter behave as though you assume the person will take care of it. But there are other situations where it is by no means clear that you have a right to expect action. The person may do as you ask, or may not. Since no one likes to be taken for granted, you need to be sensitive to the difference between these two types of situations, and tailor your requests accordingly.

One thing that almost always causes people instant annoyance is being asked to do something in a way that implies that you think it is clearly their responsibility to do it, when they feel otherwise. For example, assignment of incoming issues is particularly fertile ground for this kind of annoyance. The participants in a project usually know who is expert in what areas, so when a bug report comes in, there will often be one or two people whom everyone knows could probably fix it quickly. However, if you assign the issue over to one of those people without her prior permission, she may feel she has been put into an uncomfortable position. She senses the pressure of expectation, but also may feel that she is, in effect, being punished for her expertise. After all, the way one acquires expertise is by fixing bugs, so perhaps

someone else should take this one! (Note that issue trackers that automatically assign issues to particular people based on information in the bug report are less likely to offend, because everyone knows that the assignment was made by an automated process, and is not an indication of human expectations.)

While it would be nice to spread the load as evenly as possible, there are certain times when you just want to encourage the person who can fix a bug the fastest to do so. Given that you can't afford a communications turnaround for every such assignment ("Would you be willing to look at this bug?" "Yes." "Okay, I'm assigning the issue over to you then." "Okay."), you should simply make the assignment in the form of an inquiry, conveying no pressure. Virtually all issue trackers allow a comment to be associated with the assignment of an issue. In that comment, you can say something like this:

> Assigning this over to you, jrandom, because you're most familiar with this code. Feel free to bounce this back if you don't have time to look at it, though. (And let me know if you'd prefer not to receive such requests in the future.)

This distinguishes clearly between the *request* for assignment and the recipient's *acceptance* of that assignment. The audience here isn't only the assignee, it's everyone: the entire group sees a public confirmation of the assignee's expertise, but the message also makes it clear that the assignee is free to accept or decline the responsibility.

Follow up after you delegate

When you ask someone to do something, remember that you have done so, and follow up with him no matter what. Most requests are made in public forums, and are roughly of the form "Can you take care of X? Let us know either way; no problem if you can't, just need to know." You may or may not get a response. If you do, and the response is negative, the loop is closed—you'll need to try some other strategy for dealing with X. If there is a positive response, then keep an eye out for progress on the issue, and comment on the progress you do or don't see (everyone works better when they know someone else is appreciating their work). If there is no response after a few days, ask again, or post saying that you got no response and are looking for someone else to do it. Or just do it yourself, but still make sure to say that you got no response to the initial inquiry.

The purpose of publicly noting the lack of response is *not* to humiliate the person, and your remarks should be phrased so as not to have that effect. The purpose is simply to show that you keep track of what you have asked for, and that you notice the reactions you get. This makes people more likely to say yes next time, because they will observe (even if only unconsciously) that you are likely to notice any work they do, given that you noticed the much less visible event of someone failing to respond.

Notice what people are interested in

Another thing that makes people happy is to have their interests noticed—in general, the more aspects of someone's personality you notice and remember, the more comfortable he will be, and the more he will want to work with groups of which you are a part.

For example, there was a sharp distinction in the Subversion project between people who wanted to reach a definitive 1.0 release (which we eventually did), and people who mainly wanted to add new features and work on interesting problems but who didn't much care when 1.0 came out. Neither of these positions is better or worse than the other; they're just two different kinds of developers, and both kinds do lots of work on the project. But we swiftly learned that it was important to *not* assume that the excitement of the 1.0 drive was shared by everyone. Electronic media can be very deceptive: you may sense an atmosphere of shared purpose when, in fact, it's shared only by the people you happen to have been talking to, while others have completely different priorities.

The more aware you are of what people want out of the project, the more effectively you can make requests of them. Even just demonstrating an understanding of what they want, without making any associated request, is useful, in that it confirms to each person that she's not just another particle in an undif-

ferentiated mass.

Praise and Criticism

Praise and criticism are not opposites; in many ways, they are very similar. Both are primarily forms of attention, and are most effective when specific rather than generic. Both should be deployed with concrete goals in mind. Both can be diluted by inflation: praise too much or too often and you will devalue your praise; the same is true for criticism, though in practice, criticism is usually reactive and therefore a bit more resistant to devaluation.

An important feature of technical culture is that detailed, dispassionate criticism is often taken as a kind of praise (as discussed in the section called “Recognizing Rudeness” in Chapter 6, *Communications*), because of the implication that the recipient's work is worth the time required to analyze it. However, both of those conditions—*detailed* and *dispassionate*—must be met for this to be true. For example, if someone makes a sloppy change to the code, it is useless (and actually harmful) to follow up saying simply "That was sloppy." Sloppiness is ultimately a characteristic of a *person*, not of their work, and it's important to keep your reactions focused on the work. It's much more effective to describe all the things wrong with the change, tactfully and without malice. If this is the third or fourth careless change in a row by the same person, it's appropriate to say that—again without anger—at the end of your critique, to make it clear that the pattern has been noticed.

If someone does not improve in response to criticism, the solution is not more or stronger criticism. The solution is for the group to remove that person from the position of incompetence, in a way that minimizes hurt feelings as much as possible; see the section called “Transitions” later in this chapter for examples. That is a rare occurrence, however. Most people respond pretty well to criticism that is specific, detailed, and contains a clear (even if unspoken) expectation of improvement.

Praise won't hurt anyone's feelings, of course, but that doesn't mean it should be used any less carefully than criticism. Praise is a tool: before you use it, ask yourself *why* you want to use it. As a rule, it's not a good idea to praise people for doing what they usually do, or for actions that are a normal and expected part of participating in the group. If you were to do that, it would be hard to know when to stop: should you praise *everyone* for doing the usual things? After all, if you leave some people out, they'll wonder why. It's much better to express praise and gratitude sparingly, in response to unusual or unexpected efforts, with the intention of encouraging more such efforts. When a participant seems to have moved permanently into a state of higher productivity, adjust your praise threshold for that person accordingly. Repeated praise for normal behavior gradually becomes meaningless anyway. Instead, that person should sense that her high level of productivity is now considered normal and natural, and only work that goes beyond that should be specially noticed.

This is not to say that the person's contributions shouldn't be acknowledged, of course. But remember that if the project is set up right, everything that person does is already visible anyway, and so the group will know (and the person will know that the rest of the group knows) everything she does. There are also ways to acknowledge someone's work by means other than direct praise. You could mention in passing, while discussing a related topic, that she has done a lot of work in the given area and is the resident expert there; you could publicly consult her on some question about the code; or perhaps most effectively, you could conspicuously make further use of the work she has done, so she sees that others are now comfortable relying on the results of her work. It's probably not necessary to do these things in any calculated way. Someone who regularly makes large contributions in a project will know it, and will occupy a position of influence by default. There's usually no need to take explicit steps to ensure this, unless you sense that, for whatever reason, a contributor is underappreciated.

Prevent Territoriality

Watch out for participants who try to stake out exclusive ownership of certain areas of the project, and who seem to want to do all the work in those areas, to the extent of aggressively taking over work that others start. Such behavior may even seem healthy at first. After all, on the surface it looks like the person is taking on more responsibility, and showing increased activity within a given area. But in the long

run, it is destructive. When people sense a "no trespassing" sign, they stay away. This results in reduced review in that area, and greater fragility, because the lone developer becomes a single point of failure. Worse, it fractures the cooperative, egalitarian spirit of the project. The theory should always be that any developer is welcome to help out on any task at any time. Of course, in practice things work a bit differently: people do have areas where they are more and less influential, and non-experts frequently defer to experts in certain domains of the project. But the key is that this is all voluntary: informal authority is granted based on competence and proven judgement, but it should never be actively *taken*. Even if the person desiring the authority really is competent, it is still crucial that she hold that authority informally, through the consensus of the group, and that the authority never cause her to exclude others from working in that area.

Rejecting or editing someone's work for technical reasons is an entirely different matter, of course. There, the decisive factor is the content of the work, not who happened to act as gatekeeper. It may be that the same person happens to do most of the reviewing for a given area, but as long as he never tries to prevent someone else from doing that work too, things are probably okay.

In order to combat incipient territorialism, or even the appearance of it, many projects have taken the step of banning the inclusion of author names or designated maintainer names in source files. I wholeheartedly agree with this practice: we follow it in the Subversion project, and it is more or less official policy at the Apache Software Foundation. ASF member Sander Striker puts it this way:

> *At the Apache Software foundation we discourage the use of author tags in source code. There are various reasons for this, apart from the legal ramifications. Collaborative development is about working on projects as a group and caring for the project as a group. Giving credit is good, and should be done, but in a way that does not allow for false attribution, even by implication. There is no clear line for when to add or remove an author tag. Do you add your name when you change a comment? When you put in a one-line fix? Do you remove other author tags when you refactor the code and it looks 95% different? What do you do about people who go about touching every file, changing just enough to make the virtual author tag quota, so that their name will be everywhere?*
>
> *There are better ways to give credit, and our preference is to use those. From a technical standpoint author tags are unnecessary; if you wish to find out who wrote a particular piece of code, the version control system can be consulted to figure that out. Author tags also tend to get out of date. Do you really wish to be contacted in private about a piece of code you wrote five years ago and were glad to have forgotten?*

A software project's source code files are the core of its identity. They should reflect the fact that the developer community as a whole is responsible for them, and not be divided up into little fiefdoms.

People sometimes argue in favor of author or maintainer tags in source files on the grounds that this gives visible credit to those who have done the most work there. There are two problems with this argument. First, the tags inevitably raise the awkward question of how much work one must do to get one's own name listed there too. Second, they conflate the issue of credit with that of authority: having done work in the past does not imply ownership of the area where the work was done, but it's difficult if not impossible to avoid such an implication when individual names are listed at the tops of source files. In any case, credit information can already be obtained from the version control logs and other out-of-band mechanisms like mailing list archives, so no information is lost by banning it from the source files themselves.[2]

[2]But see the mailing list thread entitled *"having authors names in .py files"* at http://groups.google.com/group/sage-devel/browse_thread/thread/e207ce2206f0beee for a good counterargument, particularly the post from William Stein. The key in that case, I think, is that many of the authors come from a culture (the academic mathematics community) where crediting directly at the source is the norm and is highly valued. In such circumstances, it may be preferable to put author names into the source files, along with precise descriptions of what each author did, since the majority of potential contributors will expect that style of acknowledgement.

If your project decides to ban individual names from source files, make sure not to go overboard. For instance, many projects have a `contrib/` area where small tools and helper scripts are kept, often written by people who are otherwise not associated with the project. It's fine for those files to contain author names, because they are not really maintained by the project as a whole. On the other hand, if a contributed tool starts getting hacked on by other people in the project, eventually you may want to move it to a less isolated location and, assuming the original author approves, remove the author's name, so that the code looks like any other community-maintained resource. If the author is sensitive about this, compromise solutions are acceptable, for example:

```
# indexclean.py: Remove old data from a Scanley index.
#
# Original Author: K. Maru <kobayashi@yetanotheremailservice.com>
# Now Maintained By: The Scanley Project <http://www.scanley.org/>
#                    and K. Maru.
#
# ...
```

But it's better to avoid such compromises, if possible, and most authors are willing to be persuaded, because they're happy that their contribution is being made a more integral part of the project.

The important thing is to remember that there is a continuum between the core and the periphery of any project. The main source code files for the software are clearly part of the core, and should be considered as maintained by the community. On the other hand, companion tools or pieces of documentation may be the work of single individuals, who maintain them essentially alone, even though the works may be associated with, and even distributed with, the project. There is no need to apply a one-size-fits-all rule to every file, as long as the principle that community-maintained resources are not allowed to become individual territories is upheld.

The Automation Ratio

Try not to let humans do what machines could do instead. As a rule of thumb, automating a common task is worth at least ten times the effort a developer would spend doing that task manually one time. For very frequent or very complex tasks, that ratio could easily go up to twenty or even higher.

Thinking of yourself as a "project manager", rather than just another developer, may be a useful attitude here. Sometimes individual developers are too wrapped up in low-level work to see the big picture and realize that everyone is wasting a lot of effort performing automatable tasks manually. Even those who do realize it may not take the time to solve the problem: because each individual performance of the task does not feel like a huge burden, no one ever gets annoyed enough to do anything about it. What makes automation compelling is that that small burden is multiplied by the number of times each developer incurs it, and then *that* number is multiplied by the number of developers.

Here, I am using the term "automation" broadly, to mean not only repeated actions where one or two variables change each time, but any sort of technical infrastructure that assists humans. The minimum standard automation required to run a project these days was described in Chapter 3, *Technical Infrastructure*, but each project may have its own special problems too. For example, a group working on documentation might want to have a web site displaying the most up-to-date versions of the documents at all times. Since documentation is often written in a markup language like XML, there may be a compilation step, often quite intricate, involved in creating displayable or downloadable documents. Arranging a web site where such compilation happens automatically on every commit can be complicated and time-consuming—but it is worth it, even if it costs you a day or more to set up. The overall benefits of having up-to-date pages available at all times are huge, even though the cost of *not* having them might seem like only a small annoyance at any single moment, to any single developer.

Taking such steps eliminates not merely wasted time, but the griping and frustration that ensue when hu-

mans make missteps (as they inevitably will) in trying to perform complicated procedures manually. Multi-step, deterministic operations are exactly what computers were invented for; save your humans for more interesting things.

Automated testing

Automated test runs are helpful for any software project, but especially so for open source projects, because automated testing (especially regression testing) allows developers to feel comfortable changing code in areas they are unfamiliar with, and thus encourages exploratory development. Since detecting breakage is so hard to do by hand—one essentially has to guess where one might have broken something, and try various experiments to prove that one didn't—having automated ways to detect such breakage saves the project a *lot* of time. It also makes people much more relaxed about refactoring large swaths of code, and therefore contributes to the software's long-term maintainability.

> **Regression Testing**
>
> *Regression testing* means testing for the reappearance of already-fixed bugs. The purpose of regression testing is to reduce the chances that code changes will break the software in unexpected ways. As a software project gets bigger and more complicated, the chances of such unexpected side effects increase steadily. Good design can reduce the rate at which the chances increase, but it cannot eliminate the problem entirely.
>
> As a result, many projects have a *test suite*, a separate program that invokes the project's software in ways that have been known in the past to stimulate specific bugs. If the test suite succeeds in making one of these bugs happen, this is known as a *regression*, meaning that someone's change unexpectedly unfixed a previously-fixed bug.
>
> See also http://en.wikipedia.org/wiki/Regression_testing.

Regression testing is not a panacea. For one thing, it works best for programs with batch-style interfaces. Software that is operated primarily through graphical user interfaces is much harder to drive programmatically. Another problem is that the regression test suite framework itself can often be quite complex, with a learning curve and maintenance burden all its own. Reducing this complexity is one of the most useful things you can do, even though it may take a considerable amount of time. The easier it is to add new tests to the suite, the more developers will do so, and the fewer bugs will survive to release. Any effort spent making tests easier to write will be paid back manyfold over the lifetime of the project.

Many projects have a *"Don't break the build!"* rule, meaning: don't commit a change that makes the software unable to compile or run. Being the person who broke the build is usually cause for mild embarrassment and ribbing. Projects with regression test suites often have a corollary rule: don't commit any change that causes tests to fail. Such failures are easiest to spot if there are automatic nightly runs of the entire test suite, with the results mailed out to the development list or to a dedicated test-results mailing list; that's another example of a worthwhile automation.

Most volunteer developers are willing to take the extra time to write regression tests, when the test system is comprehensible and easy to work with. Accompanying changes with tests is understood to be the responsible thing to do, and it's also an easy opportunity for collaboration: often two developers will divide up the work for a bugfix, with one writing the fix itself, and the other writing the test. The latter developer may often end up with more work, and since writing a test is already less satisfying than actually fixing the bug, it is imperative that the test suite not make the experience more painful than it has to be.

Some projects go even further, requiring that a new test accompany *every* bugfix or new feature. Whether this is a good idea or not depends on many factors: the nature of the software, the makeup of the development team, and the difficulty of writing new tests. The CVS (http://www.cvshome.org/) project has long had such a rule. It is a good policy in theory, since CVS is version control software and therefore very risk-averse about the possibility of munging or mishandling the user's data. The problem in practice

is that CVS's regression test suite is a single huge shell script (amusingly named `sanity.sh`), hard to read and hard to modify or extend. The difficulty of adding new tests, combined with the requirement that patches be accompanied by new tests, means that CVS effectively discourages patches. When I used to work on CVS, I sometimes saw people start on and even complete a patch to CVS's own code, but give up when told of the requirement to add a new test to `sanity.sh`.

It is normal to spend more time writing a new regression test than on fixing the original bug. But CVS carried this phenomenon to an extreme: one might spend hours trying to design one's test properly, and still get it wrong, because there are just too many unpredictable complexities involved in changing a 35,000-line Bourne shell script. Even longtime CVS developers often grumbled when they had to add a new test.

This situation was due to a failure on all our parts to consider the automation ratio. It is true that switching to a real test framework—whether custom-built or off-the-shelf—would have been a major effort.[3] But neglecting to do so has cost the project much more, over the years. How many bugfixes and new features are *not* in CVS today, because of the impediment of an awkward test suite? We cannot know the exact number, but it is surely many times greater than the number of bugfixes or new features the developers might forgo in order to develop a new test system (or integrate an off-the-shelf system). That task would only take a finite amount of time, while the penalty of using the current test suite will continue forever if nothing is done.

The point is not that having strict requirements to write tests is bad, nor that writing your test system as a Bourne shell script is necessarily bad. It might work fine, depending on how you design it and what it needs to test. The point is simply that when the test system becomes a significant impediment to development, something must be done. The same is true for any routine process that turns into a barrier or a bottleneck.

Treat Every User as a Potential Volunteer

Each interaction with a user is an opportunity to get a new volunteer. When a user takes the time to post to one of the project's mailing lists, or to file a bug report, he has already tagged himself as having more potential for involvement than most users (from whom the project will never hear at all). Follow up on that potential: if he described a bug, thank him for the report and ask him if he wants to try fixing it. If he wrote to say that an important question was missing from the FAQ, or that the program's documentation was deficient in some way, then freely acknowledge the problem (assuming it really exists) and ask if he's interested in writing the missing material himself. Naturally, much of the time the user will demur. But it doesn't cost much to ask, and every time you do, it reminds the other listeners in that forum that getting involved in the project is something anyone can do.

Don't limit your goals to acquiring new developers and documentation writers. For example, even training people to write good bug reports pays off in the long run, if you don't spend *too* much time per person, and if they go on to submit more bug reports in the future—which they are more likely to do if they got a constructive reaction to their first report. A constructive reaction need not be a fix for the bug, although that's always the ideal; it can also be a solicitation for more information, or even just a confirmation that the behavior *is* a bug. People want to be listened to. Secondarily, they want their bugs fixed. You may not always be able to give them the latter in a timely fashion, but you (or rather, the project as a whole) can give them the former.

A corollary of this is that developers should not express anger at people who file well-intended but vague bug reports. This is one of my personal pet peeves; I see developers do it all the time on various open source mailing lists, and the harm it does is palpable. Some hapless newbie will post a useless report:

> Hi, I can't get Scanley to run. Every time I start it up, it just errors. Is anyone else seeing this problem?

[3] Note that there would be no need to convert all the existing tests to the new framework; the two could happily exist side by side, with old tests converted over only as they needed to be changed.

Some developer—who has seen this kind of report a thousand times, and hasn't stopped to think that the newbie has not—will respond like this:

> What are we supposed to do with so little information? Sheesh. Give us at least some details, like the version of Scanley, your operating system, and the error.

This developer has failed to see things from the user's point of view, and also failed to consider the effect such a reaction might have on all the *other* people watching the exchange. Naturally a user who has no programming experience, and no prior experience reporting bugs, will not know how to write a bug report. What is the right way to handle such a person? Educate them! And do it in such a way that they come back for more:

> Sorry you're having trouble. We'll need more information in order to figure out what's happening here. Please tell us the version of Scanley, your operating system, and the exact text of the error. The very best thing you can do is send a transcript showing the exact commands you ran, and the output they produced. See http://www.scanley.org/how_to_report_a_bug.html for more.

This way of responding is far more effective at extracting the needed information from the user, because it is written to the user's point of view. First, it expresses sympathy: *You had a problem; we feel your pain*. (This is not necessary in every bug report response; it depends on the severity of the problem and how upset the user seemed.) Second, instead of belittling her for not knowing how to report a bug, it tells her how, and in enough detail to be actually useful—for example, many users don't realize that "show us the error" means "show us the exact text of the error, with no omissions or abridgements." The first time you work with such a user, you need to be specific about that. Finally, it offers a pointer to much more detailed and complete instructions for reporting bugs. If you have successfully engaged with the user, she will often take the time to read that document and do what it says. This means, of course, that you have to have the document prepared in advance. It should give clear instructions about what kind of information your development team wants to see in every bug report. Ideally, it should also evolve over time in response to the particular sorts of omissions and misreports users tend to make for your project.

The Subversion project's bug reporting instructions are a fairly standard example of the form (see Appendix D, *Example Instructions for Reporting Bugs*). Notice how they close with an invitation to provide a patch to fix the bug. This is not because such an invitation will lead to a greater patch/report ratio—most users who are capable of fixing bugs already know that a patch would be welcome, and don't need to be told. The invitation's real purpose is to emphasize to all readers, especially those new to the project or new to free software in general, that the project runs on volunteer contributions. In a sense, the project's current developers are no more responsible for fixing the bug than is the person who reported it. This is an important point that many new users will not be familiar with. Once they realize it, they're more likely to help make the fix happen, if not by contributing code then by providing a more thorough reproduction recipe, or by offering to test fixes that other people post. The goal is to make every user realize that there is no *innate* difference between herself and the people who work on the project—that it's a question of how much time and effort one puts in, not a question of who one is.

The admonition against responding angrily does not apply to rude users. Occasionally people post bug reports or complaints that, regardless of their informational content, show a sneering contempt at the project for some failing. Often such people are alternately insulting and flattering, such as the person who posted this to a Subversion mailing list:

> Why is it that after almost 6 days there still aren't any binaries posted for the windows platform?!? It's the same story every time and it's pretty frustrating. Why aren't these things automated so that they could be available immediately?? When you post an "RC" build, I think the idea is that you want users to test the build, but yet you don't provide any way of doing so. Why even have a soak period if you provide no means of testing??

Initial response to this rather inflammatory post was surprisingly restrained: people pointed out that the project had a published policy of not providing official binaries, and said, with varying degrees of annoyance, that he ought to volunteer to produce them himself if they were so important to him. Believe it or not, his next post started with these lines:

> First of all, let me say that I think Subversion is awesome and I really appreciate the efforts of everyone involved. [...]

...and then he went on to berate the project *again* for not providing binaries, while still not volunteering to do anything about it. After that, about 50 people just jumped all over him, and I can't say I really minded. The "zero-tolerance" policy toward rudeness advocated in the section called “Nip Rudeness in the Bud” in Chapter 2, *Getting Started* applies to people with whom the project has (or would like to have) a sustained interaction. But when someone makes it clear from the start that he is going to be a fountain of bile, there is no point making him feel welcome.

Such situations are fortunately quite rare, and they are noticeably rarer in projects that make an effort to engage users constructively and courteously from their very first interaction.

Share Management Tasks as Well as Technical Tasks

Share the management burden as well as the technical burden of running the project. As a project becomes more complex, more and more of the work is about managing people and information flow. There is no reason not to share that burden, and sharing it does not necessarily require a top-down hierarchy either—what happens in practice tends to be more of a peer-to-peer network topology than a military-style command structure.

Sometimes management roles are formalized, and sometimes they happen spontaneously. In the Subversion project, we have a patch manager, a translation manager, documentation managers, issue managers (albeit unofficial), and a release manager. Some of these roles we made a conscious decision to initiate, others just happened by themselves; as the project grows, I expect more roles to be added. Below we'll examine these roles, and a couple of others, in detail (except for release manager, which was already covered in the section called “Release manager” and the section called “Dictatorship by Release Owner” earlier in this chapter).

As you read the role descriptions, notice that none of them requires exclusive control over the domain in question. The issue manager does not prevent other people from making changes in the issues database, the FAQ manager does not insist on being the only person to edit the FAQ, and so on. These roles are all about responsibility without monopoly. An important part of each domain manager's job is to notice when other people are working in that domain, and train them to do the things the way the manager does, so that the multiple efforts reinforce rather than conflict. Domain managers should also document the processes by which they do their work, so that when one leaves, someone else can pick up the slack right away.

Sometimes there is a conflict: two or more people want the same role. There is no one right way to handle this. You could suggest that each volunteer post a proposal (an "application") and have all the committers vote on which is best. But this is cumbersome and potentially awkward. I find that a better technique is just to ask the multiple candidates to settle it among themselves. They usually will, and will be more satisfied with the result than if a decision had been imposed on them from the outside.

Patch Manager

In a free software project that receives a lot of patches, keeping track of which patches have arrived and what has been decided about them can be a nightmare, especially if done in a decentralized way. Most

patches arrive as posts to the project's development mailing list (though some may appear first in the issue tracker, or on external web sites), and there are a number of different routes a patch can take after arrival.

Sometimes someone reviews the patch, finds problems, and bounces it back to the original author for cleanup. This usually leads to an iterative process—all visible on the mailing list—in which the original author posts revised versions of the patch until the reviewer has nothing more to criticize. It is not always easy to tell when this process is done: if the reviewer commits the patch, then clearly the cycle is complete. But if she does not, it might be because she simply didn't have time, or doesn't have commit access herself and couldn't rope any of the other developers into doing it.

Another frequent response to a patch is a freewheeling discussion, not necessarily about the patch itself, but about whether the concept behind the patch is good. For example, the patch may fix a bug, but the project prefers to fix that bug in another way, as part of solving a more general class of problems. Often this is not known in advance, and it is the patch that stimulates the discovery.

Occasionally, a posted patch is met with utter silence. Usually this is due to no developer having time *at that moment* to review the patch, so each hopes that someone else will do it. Since there's no particular limit to how long each person waits for someone else to pick up the ball, and meanwhile other priorities are always coming up, it's very easy for a patch to slip through the cracks without any single person intending for that to happen. The project might miss out on a useful patch this way, and there are other harmful side effects as well: it is discouraging to the author, who invested work in the patch, and it makes the project as a whole look a bit out of touch, especially to others considering writing patches.

The patch manager's job is to make sure that patches don't "slip through the cracks." This is done by following every patch through to some sort of stable state. The patch manager watches every mailing list thread that results from a patch posting. If it ends in a commit of the patch, he does nothing. If it goes into a review/revise iteration, ending with a final version of the patch but no commit, he files an issue pointing to the final version, and to the mailing list thread around it, so that there is a permanent record for developers to follow up on later. If the patch addresses an existing issue, he annotates that issue with the relevant information, instead of opening a new issue.

When a patch gets no reaction at all, the patch manager waits a few days, then follows up asking if anyone is going to review it. This usually gets a reaction: a developer may explain that she doesn't think the patch should be applied, and give the reasons why, or she may review it, in which case one of the previously described paths is taken. If there is still no response, the patch manager may or may not file an issue for the patch, at his discretion, but at least the original submitter got *some* reaction.

Having a patch manager has saved the Subversion development team a lot of time and mental energy. Without a designated person to take responsibility, every developer would constantly have to worry "If I don't have time to respond to this patch right now, can I count on someone else doing it? Should I try to keep an eye on it? But if other people are also keeping an eye on it, for the same reasons, then we'd have needlessly duplicated effort." The patch manager removes the second-guessing from the situation. Each developer can make the decision that is right for her at the moment she first sees the patch. If she wants to follow up with a review, she can do that—the patch manager will adjust his behavior accordingly. If she wants to ignore the patch completely, that's fine too; the patch manager will make sure it isn't forgotten.

Because this system works only if people can depend on the patch manager being there without fail, the role should be held formally. In Subversion, we advertised for it on the development and users mailing lists, got several volunteers, and took the first one who replied. When that person had to step down (see the section called “Transitions” later in this chapter), we did the same thing again. We've never tried having multiple people share the role, because of the communications overhead that would be required between them; but perhaps at very high volumes of patch submission, a multiheaded patch manager might make sense.

Translation Manager

In software projects, "translation" can refer to two very different things. It can mean translating the software's documentation into other languages, or it can mean translating the software itself—that is, having the program display errors and help messages in the user's preferred language. Both are complex tasks, but once the right infrastructure is in place, they are largely separable from other development. Because the tasks are similar in some ways, it may make sense (depending on your project) to have a single translation manager handle both, or it may be better to have two different managers.

In the Subversion project, we have one translation manager handle both. He does not actually write the translations himself, of course—he may help out on one or two, but as of this writing, he would need to speak ten languages (twelve counting dialects) in order to work on all of them! Instead, he manages teams of volunteer translators: he helps them coordinate among each other, and he coordinates between the teams and the rest of the project.

Part of the reason the translation manager is necessary is that translators are a different demographic from developers. They sometimes have little or no experience working in a version control repository, or indeed with working as part of a distributed volunteer team at all. But in other respects they are often the best kind of volunteer: people with specific domain knowledge who saw a need and chose to get involved. They are usually willing to learn, and enthusiastic to get to work. All they need is someone to tell them how. The translation manager makes sure that the translations happen in a way that does not interfere unnecessarily with regular development. He also serves as a sort of representative of the translators as a unified body, whenever the developers must be informed of technical changes required to support the translation effort.

Thus, the position's most important skills are diplomatic, not technical. For example, in Subversion we have a policy that all translations should have at least two people working on them, because otherwise there is no way for the text to be reviewed. When a new volunteer shows up offering to translate Subversion to, say, Malagasy, the translation manager has to either hook him up with someone who posted six months ago expressing interest in doing a Malagasy translation, or else politely ask the volunteer to go find *another* Malagasy translator to work with as a partner. Once enough people are available, the manager sets them up with the proper kind of commit access, informs them of the project's conventions (such as how to write log messages), and then keeps an eye out to make sure they adhere to those conventions.

Conversations between the translation manager and the developers, or between the translation manager and translation teams, are usually held in the project's original language—that is, the language from which all the translations are being made. For most free software projects, this is English, but it doesn't matter what it is as long as the project agrees on it. (English is probably best for projects that want to attract a broad international development community, though.)

Conversations *within* a particular translation team usually happen in their shared language, however, and one of the other tasks of the translation manager is to set up a dedicated mailing list for each team. That way the translators can discuss their work freely, without distracting people on the project's main lists, most of whom would not be able to understand the translation language anyway.

Internationalization Versus Localization

Internationalization (*I18N*) and *localization* (*L10N*) both refer to the process of adapting a program to work in linguistic and cultural environments other than the one for which it was originally written. The terms are often treated as interchangeable, but in fact they are not quite the same thing. As http://en.wikipedia.org/wiki/G11n writes:

> The distinction between them is subtle but important: Internationalization is the adaptation of products for *potential* use virtually everywhere, while localization is the addition of special features for use in a *specific* locale.

For example, changing your software to losslessly handle Unicode (ht-

tp://en.wikipedia.org/wiki/Unicode) text encodings is an internationalization move, since it's not about a particular language, but rather about accepting text from any of a number of languages. On the other hand, making your software print all error messages in Slovenian, when it detects that it is running in a Slovenian environment, is a localization move.

Thus, the translation manager's task is principally about localization, not internationalization.

Documentation Manager

Keeping software documentation up-to-date is a never-ending task. Every new feature or enhancement that goes into the code has the potential to cause a change in the documentation. Also, once the project's documentation reaches a certain level of completeness, you will find that a lot of the patches people send in are for the documentation, not for the code. This is because there are many more people competent to fix bugs in prose than in code: all users are readers, but only a few are programmers.

Documentation patches are usually much easier to review and apply than code patches. There is little or no testing to be done, and the quality of the change can be evaluated quickly just by review. Since the quantity is high, but the review burden fairly low, the ratio of administrative overhead to productive work is greater for documentation patches than for code patches. Furthermore, most of the patches will probably need some sort of adjustment, in order to maintain a consistent authorial voice in the documentation. In many cases, patches will overlap with or affect other patches, and need to be adjusted with respect to each other before being committed.

Given the exigencies of handling documentation patches, and the fact that the code base needs to be constantly monitored so the documentation can be kept up-to-date, it makes sense to have one person, or a small team, dedicated to the task. They can keep a record of exactly where and how the documentation lags behind the software, and they can have practiced procedures for handling large quantities of patches in an integrated way.

Of course, this does not preclude other people in the project from applying documentation patches on the fly, especially small ones, as time permits. And the same patch manager (see the section called “Patch Manager” earlier in this chapter) can track both code and documentation patches, filing them wherever the development and documentation teams want them, respectively. (If the total quantity of patches ever exceeds one human's capacity to track, though, switching to separate patch managers for code and documentation is probably a good first step.) The point of a documentation team is to have people who think of themselves as responsible for keeping the documentation organized, up-to-date, and consistent with itself. In practice, this means knowing the documentation intimately, watching the code base, watching the changes *others* commit to the documentation, watching for incoming documentation patches, and using all these information sources to do whatever is necessary to keep the documentation healthy.

Issue Manager

The number of issues in a project's bug tracker grows in proportion to the number of people using the software. Therefore, even as you fix bugs and ship an increasingly robust program, you should still expect the number of open issues to grow essentially without bound. The frequency of duplicate issues will also increase, as will the frequency of incomplete or poorly described issues.

Issue managers help alleviate these problems by watching what goes into the database, and periodically sweeping through it looking for specific problems. Their most common action is probably to fix up incoming issues, either because the reporter didn't set some of the form fields correctly, or because the issue is a duplicate of one already in the database. Obviously, the more familiar an issue manager is with the project's bug database, the more efficiently she will be able to detect duplicate issues—this is one of the main advantages of having a few people specialize in the bug database, instead of everyone trying to do it *ad hoc*. When the group tries to do it in a decentralized manner, no single individual acquires a

deep expertise in the content of the database.

Issue managers can also help map between issues and individual developers. When there are a lot of bug reports coming in, not every developer may read the issue notification mailing list with equal attention. However, if someone who knows the development team is keeping an eye on all incoming issues, then she can discreetly direct certain developers' attention to specific bugs when appropriate. Of course, this has to be done with a sensitivity to everything else going on in development, and to the recipient's desires and temperament. Therefore, it is often best for issue managers to be developers themselves.

Depending on how your project uses the issue tracker, issue managers can also shape the database to reflect the project's priorities. For example, in Subversion we schedule issues into specific future releases, so that when someone asks "When will bug X be fixed?" we can say "Two releases from now," even if we can't give an exact date. The releases are represented in the issue tracker as target milestones, a field available in IssueZilla.[4] As a rule, every Subversion release has one major new feature and a list of specific bug fixes. We assign the appropriate target milestone to all the issues planned for that release (including the new feature—it gets an issue too), so that people can view the bug database through the lens of release scheduling. These targets rarely remain static, however. As new bugs come in, priorities sometimes get shifted around, and issues must be moved from one milestone to another so that each release remains manageable. This, again, is best done by people who have an overall sense of what's in the database, and how various issues relate to each other.

Another thing issue managers do is notice when issues become obsolete. Sometimes a bug is fixed accidentally as part of an unrelated change to the software, or sometimes the project changes its mind about whether a certain behavior is buggy. Finding obsoleted issues is not easy: the only way to do it systematically is by making a sweep over all the issues in the database. Full sweeps become less and less feasible over time, however, as the number of issues grows. After a certain point, the only way to keep the database sane is to use a divide-and-conquer approach: categorize issues immediately on arrival and direct them to the appropriate developer's or team's attention. The recipient then takes charge of the issue for the rest of its lifetime, shepherding it to resolution or oblivion as necessary. When the database is that large, the issue manager becomes more of an overall coordinator, spending less time looking at each issue herself and more time getting it into the right person's hands.

FAQ Manager

FAQ maintenance is a surprisingly difficult problem. Unlike most other documents in a project, whose content is planned out in advance by the authors, a FAQ is a wholly reactive document (see Maintaining a FAQ). No matter how big it gets, you still never know what the next addition will be. And because it is always added to piecemeal, it is very easy for the document as a whole to become incoherent and disorganized, and even to contain duplicate or semi-duplicate entries. Even when it does not have any obvious problems like that, there are often unnoticed interdependencies between items—links that should be made but aren't—because the related items were added a year apart.

The role of a FAQ manager is twofold. First, she maintains the overall quality of the FAQ by staying familiar with at least the topics of all the questions in it, so that when people add new items that are duplicates of, or related to, existing items, the appropriate adjustments can be made. Second, she watches the project mailing lists and other forums for recurring problems or questions, and to write new FAQ entries based on this input. This latter task can be quite complex: one must be able to follow a thread, recognize the core questions raised in it, post a proposed FAQ entry, incorporate comments from others (since it's impossible for the FAQ manager to be an expert in every topic covered by the FAQ), and sense when the process is finished so the item can at last be added.

The FAQ manager usually also becomes the default expert in FAQ formatting. There are a lot of little details involved in keeping a FAQ in shape (see the section called “Treat all resources like archives” in Chapter 6, *Communications*); when random people edit the FAQ, they will sometimes forget some of these details. That's okay, as long as the FAQ manager is there to clean up after them.

[4] IssueZilla is the issue tracker we use; it is a descendant of BugZilla.

Various free software is available to help with the process of FAQ maintenance. It's fine to use it, as long as it doesn't compromise the quality of the FAQ, but beware of over-automation. Some projects try to fully automate the process of FAQ maintenance, allowing everyone to contribute and edit FAQ items in a manner similar to a wiki (see the section called "Wikis" in Chapter 3, *Technical Infrastructure*). I've seen this happen particularly with Faq-O-Matic (http://faqomatic.sourceforge.net/), though it may be that the cases I saw were simply abuses that went beyond what Faq-O-Matic was originally intended for. In any case, while complete decentralization of FAQ maintenance does reduce the workload for the project, it also results in a poorer FAQ. There's no one person with a broad view of the entire FAQ, no one to notice when certain items need updating or become obsolete entirely, and no one keeping watch for interdependencies between items. The result is a FAQ that often fails to provide users what they were looking for, and in the worst cases misleads them. Use whatever tools you need to to maintain your project's FAQ, but never let the convenience of the tools seduce you into compromising the quality of the FAQ.

See Sean Michael Kerner's article, *The FAQs on FAQs*, at http://osdir.com/Article1722.phtml, for descriptions and evaluations of open source FAQ maintenance tools.

Transitions

From time to time, a volunteer in a position of ongoing responsibility (e.g., patch manager, translation manager, etc.) will become unable to perform the duties of the position. It may be because the job turned out to be more work than he anticipated, or it may be due to completely external factors: marriage, a new baby, a new employer, or whatever.

When a volunteer gets swamped like this, he usually doesn't notice it right away. It happens by slow degrees, and there's no point at which he consciously realizes that he can no longer fulfill the duties of the role. Instead, the rest of the project just doesn't hear much from him for a while. Then there will suddenly be a flurry of activity, as he feels guilty for neglecting the project for so long and sets aside a night to catch up. Then you won't hear from him for a while longer, and then there might or might not be another flurry. But there's rarely an unsolicited formal resignation. The volunteer was doing the job in his spare time, so resigning would mean openly acknowledging to himself that his spare time is permanently reduced. People are often reluctant to do that.

Therefore, it's up to you and the others in the project to notice what's happening—or rather, not happening—and to ask the volunteer what's going on. The inquiry should be friendly and 100% guilt-free. Your purpose is to find out a piece of information, not to make the person feel bad. Generally, the inquiry should be visible to the rest of the project, but if you know of some special reason why a private inquiry would be better, that's fine too. The main reason to do it publicly is so that if the volunteer responds by saying that he won't be able to do the job anymore, there's a context established for your *next* public post: a request for a new volunteer to fill that role.

Sometimes, a volunteer is unable to do the job he's taken on, but is either unaware or unwilling to admit that fact. Of course, anyone may have trouble at first, especially if the responsibility is complex. However, if someone just isn't working out in the task he's taken on, even after everyone else has given all the help and suggestions they can, then the only solution is for him to step aside and let someone new have a try. And if the person doesn't see this himself, he'll need to be told. There's basically only one way to handle this, I think, but it's a multistep process and each step is important.

First, make sure you're not crazy. Privately talk to others in the project to see if they agree that the problem is as serious as you think it is. Even if you're already positive, this serves the purpose of letting others know that you're considering asking the person to step aside. Usually no one will object to that—they'll just be happy you're taking on the awkward task, so they don't have to!

Next, *privately* contact the volunteer in question and tell him, kindly but directly, about the problems you see. Be specific, giving as many examples as possible. Make sure to point out how people had tried to help, but that the problems persisted without improving. You should expect this email to take a long time to write, but with this sort of message, if you don't back up what you're saying, you shouldn't say it at all. Say that you would like to find a new volunteer to fill the role, but also point out that there are

many other ways to contribute to the project. At this stage, don't say that you've talked to others about it; nobody likes to be told that people were conspiring behind his back.

There are a few different ways things can go after that. The most likely reaction is that he'll agree with you, or at any rate not want to argue, and be willing to step down. In that case, suggest that he make the announcement himself, and then you can follow up with a post seeking a replacement.

Or, he may agree that there have been problems, but ask for a little more time (or for one more chance, in the case of discrete-task roles like release manager). How you react to that is a judgement call, but whatever you do, don't agree to it just because you feel like you can't refuse such a reasonable request. That would prolong the agony, not lessen it. There is often a very good reason to refuse the request, namely, that there have already been plenty of chances, and that's how things got to where they are now. Here's how I put it in a mail to someone who was filling the release manager role but was not really suited for it:

```
> If you wish to replace me with some one else, I will gracefully
> pass on the role to who comes next.  I have one request, which
> I hope is not unreasonable.  I would like to attempt one more
>  release in an effort to prove myself.

I totally understand the desire (been there myself!), but in
this case, we shouldn't do the "one more try" thing.

This isn't the first or second release, it's the sixth or
seventh... And for all of those, I know you've been dissatisfied
with the results too (because we've talked about it before).  So
we've effectively already been down the one-more-try route.
Eventually, one of the tries has to be the last one... I think
[this past release] should be it.
```

In the worst case, the volunteer may disagree outright. Then you have to accept that things are going to be awkward and plow ahead anyway. Now is the time to say that you talked to other people about it (but still don't say who until you have their permission, since those conversations were confidential), and that you don't think it's good for the project to continue as things are. Be insistent, but never threatening. Keep in mind that with most roles, the transition really happens the moment someone new starts doing the job, *not* the moment the old person stops doing it. For example, if the contention is over the role of, say, issue manager, at any point you and other influential people in the project can solicit for a new issue manager. It's not actually necessary that the person who was previously doing it stop doing it, as long as he does not sabotage (deliberately or otherwise) the efforts of the new volunteer.

Which leads to a tempting thought: instead of asking the person to resign, why not just frame it as a matter of getting him some help? Why not just have two issue managers, or patch managers, or whatever the role is?

Although that may sound nice in theory, it is generally not a good idea. What makes the manager roles work—what makes them useful, in fact—is their centralization. Those things that can be done in a decentralized fashion are usually already being done that way. Having two people fill one managerial role introduces communications overhead between those two people, as well as the potential for slippery displacement of responsibility ("I thought you brought the first aid kit!" "Me? No, I thought *you* brought the first aid kit!"). Of course, there are exceptions. Sometimes two people work extremely well together, or the nature of the role is such that it can easily be spread across multiple people. But these are not likely to be of much use when you see someone flailing in a role he is not suited for. If he'd appreciated the problem in the first place, he would have sought such help before now. In any case, it would be disrespectful to let someone waste time continuing to do a job no one will pay attention to.

The most important factor in asking someone to step down is privacy: giving him the space to make a decision without feeling like others are watching and waiting. I once made the mistake—an obvious mistake, in retrospect—of mailing all three parties at once in order to ask Subversion's release manager

to step aside in favor of two other volunteers. I'd already talked to the two new people privately, and knew that they were willing to take on the responsibility. So I thought, naïvely and somewhat insensitively, that I'd save some time and hassle by sending one mail to all of them to initiate the transition. I assumed that the current release manager was already fully aware of the problems and would see the reasonableness of my point immediately.

I was wrong. The current release manager was very offended, and rightly so. It's one thing to be asked to hand off the job; it's another thing to be asked that *in front of* the people you'll hand it off to. Once I got it through my head why he was offended, I apologized. He eventually did step aside gracefully, and continues to be involved with the project today. But his feelings were hurt, and needless to say, this was not the most auspicious of beginnings for the new volunteers either.

Committers

As the only formally distinct class of people found in all open source projects, committers deserve special attention here. Committers are an unavoidable concession to discrimination in a system which is otherwise as non-discriminatory as possible. But "discrimination" is not meant as a pejorative here. The function committers perform is utterly necessary, and I do not think a project could succeed without it. Quality control requires, well, control. There are always many people who feel competent to make changes to a program, and some smaller number who actually are. The project cannot rely on people's own judgement; it must impose standards and grant commit access only to those who meet them[5]. On the other hand, having people who can commit changes directly working side-by-side with people who cannot sets up an obvious power dynamic. That dynamic must be managed so that it does not harm the project.

In the section called “Who Votes?” in Chapter 4, *Social and Political Infrastructure*, we already discussed the mechanics of considering new committers. Here we will look at the standards by which potential new committers should be judged, and how this process should be presented to the larger community.

Choosing Committers

In the Subversion project, we choose committers primarily on the Hippocratic Principle: *first, do no harm*. Our main criterion is not technical skill or even knowledge of the code, but merely that the committer show good judgement. Judgement can mean simply knowing what not to take on. A person might post only small patches, fixing fairly simple problems in the code; but if the patches apply cleanly, do not contain bugs, and are mostly in accord with the project's log message and coding conventions, and there are enough patches to show a clear pattern, then an existing committer will usually propose that person for commit access. If at least three people say yes, and no one objects, then the offer is made. True, we might have no evidence that the person is able to solve complex problems in all areas of the code base, but that does not matter: the person has made it clear that he is capable of at least judging his own abilities. Technical skills can be learned (and taught), but judgement, for the most part, cannot. Therefore, it is the one thing you want to make sure a person has before you give him commit access.

When a new committer proposal does provoke a discussion, it is usually not about technical ability, but rather about the person's behavior on the mailing lists or in IRC. Sometimes someone shows technical skill and an ability to work within the project's formal guidelines, yet is also consistently belligerent or uncooperative in public forums. That's a serious concern; if the person doesn't seem to shape up over time, even in response to hints, then we won't add him as a committer no matter how skilled he is. In a volunteer group, social skills, or the ability to "play well in the sandbox", are as important as raw tech-

[5]Note that the commit access means something a bit different in decentralized version control systems, where anyone can set up a repository that is linked into the project, and give themselves commit access to that repository. Nevertheless, the *concept* of commit access still applies: "commit access" is shorthand for "the right to make changes to the code that will ship in the group's next release of the software." In centralized version control systems, this means having direct commit access; in decentralized ones, it means having one's changes pulled into the main distribution by default. It is the same idea either way; the mechanics by which it is realized are not terribly important.

nical ability. Because everything is under version control, the penalty for adding a committer you shouldn't have is not so much the problems it could cause in the code (review would spot those quickly anyway), but that it might eventually force the project to revoke the person's commit access—an action that is never pleasant and can sometimes be confrontational.

Many projects insist that the potential committer demonstrate a certain level of technical expertise and persistence, by submitting some number of nontrivial patches—that is, not only do these projects want to know that the person will do no harm, they want to know that she is likely to do good across the code base. This is fine, but be careful that it doesn't start to turn committership into a matter of membership in an exclusive club. The question to keep in everyone's mind should be "What will bring the best results for the code?" not "Will we devalue the social status associated with committership by admitting this person?" The point of commit access is not to reinforce people's self-worth, it's to allow good changes to enter the code with a minimum of fuss. If you have 100 committers, 10 of whom make large changes on a regular basis, and the other 90 of whom just fix typos and small bugs a few times a year, that's still better than having only the 10.

Revoking Commit Access

The first thing to be said about revoking commit access is: try not to be in that situation in the first place. Depending on whose access is being revoked, and why, the discussions around such an action can be very divisive. Even when not divisive, they will be a time-consuming distraction from productive work.

However, if you must do it, the discussion should be had privately among the same people who would be in a position to vote for *granting* that person whatever flavor of commit access they currently have. The person herself should not be included. This contradicts the usual injunction against secrecy, but in this case it's necessary. First, no one would be able to speak freely otherwise. Second, if the motion fails, you don't necessarily want the person to know it was ever considered, because that could open up questions ("Who was on my side? Who was against me?") that lead to the worst sort of factionalism. In certain rare circumstances, the group may want someone to know that revocation of commit access is or was being considered, as a warning, but this openness should be a decision the group makes. No one should ever, on her own initiative, reveal information from a discussion and ballot that others assumed were secret.

Once someone's access is revoked, that fact is unavoidably public (see the section called “Avoid Mystery” later in this chapter), so try to be as tactful as you can in how it is presented to the outside world.

Partial Commit Access

Some projects offer gradations of commit access. For example, there might be contributors whose commit access gives them free rein in the documentation, but who do not commit to the code itself. Common areas for partial commit access include documentation, translations, binding code to other programming languages, specification files for packaging (e.g., RedHat RPM spec files, etc.), and other places where a mistake will not result in a problem for the core project.

Since commit access is not only about committing, but about being part of an electorate (see the section called “Who Votes?” in Chapter 4, *Social and Political Infrastructure*), the question naturally arises: what can the partial committers vote on? There is no one right answer; it depends on what sorts of partial commit domains your project has. In Subversion we've kept things fairly simple: a partial committer can vote on matters confined exclusively to that committer's domain, and not on anything else. Importantly, we do have a mechanism for casting advisory votes (essentially, the committer writes "+0" or "+1 (non-binding)" instead of just "+1" on the ballot). There's no reason to silence people entirely just because their vote isn't formally binding.

Full committers can vote on anything, just as they can commit anywhere, and only full committers vote on adding new committers of any kind. In practice, though, the ability to add new partial committers is usually delegated: any full committer can "sponsor" a new partial committer, and partial committers in a domain can often essentially choose new committers for that same domain (this is especially helpful in

making translation work run smoothly).

Your project may need a slightly different arrangement, depending on the nature of the work, but the same general principles apply to all projects. Each committer should be able to vote on matters that fall within the scope of her commit access, and not on matters outside that, and votes on procedural questions should default to the full committers, unless there's some reason (as decided by the full committers) to widen the electorate.

Regarding enforcement of partial commit access: it's often best *not* to have the version control system enforce partial commit domains, even if it can. See the section called “Authorization” in Chapter 3, *Technical Infrastructure* for the reasons why.

Dormant Committers

Some projects automatically remove people's commit access if they go a certain amount of time (say, a year) without committing anything. I think this is usually unhelpful and even counterproductive, for two reasons.

First, it may tempt some people into committing acceptable but unnecessary changes, just to prevent their commit access from expiring. Second, it doesn't really serve any purpose. If the main criterion for granting commit access is good judgement, then why assume someone's judgement would deteriorate just because he's away from the project for a while? Even if he completely vanishes for years, not looking at the code or following development discussions, when he reappears he'll *know* how out of touch he is, and act accordingly. You trusted his judgement before, so why not trust it always? If high school diplomas do not expire, then commit access certainly shouldn't.

Sometimes a committer may ask to be removed, or to be explicitly marked as dormant in the list of committers (see the section called “Avoid Mystery” below for more about that list). In these cases, the project should accede to the person's wishes, of course.

Avoid Mystery

Although the discussions around adding any particular new committer must be confidential, the rules and procedures themselves need not be secret. In fact, it's best to publish them, so people realize that the committers are not some mysterious Star Chamber, closed off to mere mortals, but that anyone can join simply by posting good patches and knowing how to handle herself in the community. In the Subversion project, we put this information right in the developer guidelines document, since the people most likely to be interested in how commit access is granted are those thinking of contributing code to the project.

In addition to publishing the procedures, publish the actual *list* of committers. The traditional place for this is a file called `MAINTAINERS` or `COMMITTERS` in the top level of the project's source code tree. It should list all the full committers first, followed by the various partial commit domains and the members of each domain. Each person should be listed by name and email address, though the address can be encoded to prevent spam (see the section called “Address hiding in archives” in Chapter 3, *Technical Infrastructure*) if the person prefers that.

Since the distinction between full commit and partial commit access is obvious and well defined, it is proper for the list to make that distinction too. Beyond that, the list should not try to indicate the informal distinctions that inevitably arise in a project, such as who is particularly influential and how. It is a public record, not an acknowledgments file. List committers either in alphabetical order, or in the order in which they arrived.

Credit

Credit is the primary currency of the free software world. Whatever people may say about their motivations for participating in a project, I don't know any developers who would be happy doing all their work

anonymously, or under someone else's name. There are tangible reasons for this: one's reputation in a project roughly governs how much influence one has, and participation in an open source project can also indirectly have monetary value, because some employers now look for it on resumés. There are also intangible reasons, perhaps even more powerful: people simply want to be appreciated, and instinctively look for signs that their work was recognized by others. The promise of credit is therefore one of best motivators the project has. When small contributions are acknowledged, people come back to do more.

One of the most important features of collaborative development software (see Chapter 3, *Technical Infrastructure*) is that it keeps accurate records of who did what, when. Wherever possible, use these existing mechanisms to make sure that credit is distributed accurately, and be specific about the nature of the contribution. Don't just write "Thanks to J. Random <jrandom@example.com>" if instead you can write "Thanks to J. Random <jrandom@example.com> for the bug report and reproduction recipe" in a log message.

In Subversion, we have an informal but consistent policy of crediting the reporter of a bug in either the issue filed, if there is one, or the log message of the commit that fixes the bug, if not. A quick survey of Subversion commit logs up to commit number 14525 shows that about 10% of commits give credit to someone by name and email address, usually the person who reported or analyzed the bug fixed by that commit. Note that this person is different from the developer who actually made the commit, whose name is already recorded automatically by the version control system. Of the 80-odd full and partial committers Subversion has today, 55 were credited in the commit logs (usually multiple times) before they became committers themselves. This does not, of course, prove that being credited was a factor in their continued involvement, but it at least sets up an atmosphere in which people know they can count on their contributions being acknowledged.

It is important to distinguish between routine acknowledgment and special thanks. When discussing a particular piece of code, or some other contribution someone made, it is fine to acknowledge their work. For example, saying "Daniel's recent changes to the delta code mean we can now implement feature X" simultaneously helps people identify which changes you're talking about and acknowledges Daniel's work. On the other hand, posting solely to thank Daniel for the delta code changes serves no immediate practical purpose. It doesn't add any information, since the version control system and other mechanisms have already recorded the fact that he made the changes. Thanking everyone for everything would be distracting and ultimately information-free, since thanks are effective largely by how much they stand out from the default, background level of favorable comment going on all the time. This does not mean, of course, that you should never thank people. Just make sure to do it in ways that tend not to lead to credit inflation. Following these guidelines will help:

- The more ephemeral the forum, the more free you should feel to express thanks there. For example, thanking someone for their bugfix in passing during an IRC conversation is fine, as is an aside in an email devoted mainly to other topics. But don't post an email solely to thank someone, unless it's for a truly unusual feat. Likewise, don't clutter the project's web pages with expressions of gratitude. Once you start that, it'll never be clear when or where to stop. And *never* put thanks into comments in the code; that would only be a distraction from the primary purpose of comments, which is to help the reader understand the code.

- The less involved someone is in the project, the more appropriate it is to thank her for something she did. This may sound counterintuitive, but it fits with the attitude that expressing thanks is something you do when someone contributes even more than you thought she would. Thus, to constantly thank regular contributors for doing what they normally do would be to express a lower expectation of them than they have of themselves. If anything, you want to aim for the opposite effect!

  There are occasional exceptions to this rule. It's acceptable to thank someone for fulfilling his expected role when that role involves temporary, intense efforts from time to time. The canonical example is the release manager, who goes into high gear around the time of each release, but otherwise lies dormant (dormant as a release manager, in any case—he may also be an active developer, but that's a different matter).

- As with criticism and crediting, gratitude should be specific. Don't thank people just for being great, even if they are. Thank them for something they did that was out of the ordinary, and for bonus points, say exactly why what they did was so great.

In general, there is always a tension between making sure that people's individual contributions are recognized, and making sure the project is a group effort rather than a collection of individual glories. Just remain aware of this tension and try to err on the side of group, and things won't get out of hand.

Forks

In the section called “Forkability” in Chapter 4, *Social and Political Infrastructure*, we saw how the *potential* to fork has important effects on how projects are governed. But what happens when a fork actually occurs? How should you handle it, and what effects can you expect it to have? Conversely, when should you *initiate* a fork?

The answers depend on what kind of fork it is. Some forks are due to amicable but irreconcilable disagreements about the direction of the project; perhaps more are due to both technical disagreements and interpersonal conflicts. Of course, it's not always possible to tell the difference between the two, as technical arguments may involve personal elements as well. What all forks have in common is that one group of developers (or sometimes even just one developer) has decided that the costs of working with some or all of the others now outweigh the benefits.

Once a project forks, there is no definitive answer to the question of which fork is the "true" or "original" project. People will colloquially talk of fork F coming out of project P, as though P is continuing unchanged down some natural path while F diverges into new territory, but this is, in effect, a declaration of how that particular observer feels about it. It is fundamentally a matter of perception: when a large enough percentage of observers agree, the assertion starts to become true. It is not the case that there is an objective truth from the outset, one that we are only imperfectly able to perceive at first. Rather, the perceptions *are* the objective truth, since ultimately a project—or a fork—is an entity that exists only in people's minds anyway.

If those initiating the fork feel that they are sprouting a new branch off the main project, the perception question is resolved immediately and easily. Everyone, both developers and users, will treat the fork as a new project, with a new name (perhaps based on the old name, but easily distinguishable from it), a separate web site, and a separate philosophy or goal. Things get messier, however, when both sides feel they are the legitimate guardians of the original project and therefore have the right to continue using the original name. If there is some organization with trademark rights to the name, or legal control over the domain or web pages, that usually resolves the issue by fiat: that organization will decide who is the project and who is the fork, because it holds all the cards in a public relations war. Naturally, things rarely get that far: since everyone already knows what the power dynamics are, they will avoid fighting a battle whose outcome is known in advance, and just jump straight to the end.

Fortunately, in most cases there is little doubt as to which is the project and which is the fork, because a fork is, in essence, a vote of confidence. If more than half of the developers are in favor of whatever course the fork proposes to take, usually there is no need to fork—the project can simply go that way itself, unless it is run as a dictatorship with a particularly stubborn dictator. On the other hand, if fewer than half of the developers are in favor, the fork is a clearly minority rebellion, and both courtesy and common sense indicate that it should think of itself as the divergent branch rather than the main line.

Handling a Fork

If someone threatens a fork in your project, keep calm and remember your long-term goals. The mere *existence* of a fork isn't what hurts a project; rather, it's the loss of developers and users. Your real aim, therefore, is not to squelch the fork, but to minimize these harmful effects. You may be mad, you may feel that the fork was unjust and uncalled for, but expressing that publicly can only alienate undecided

developers. Instead, don't force people to make exclusive choices, and be as cooperative as is practicable with the fork. To start with, don't remove someone's commit access in your project just because he decided to work on the fork. Work on the fork doesn't mean that person has suddenly lost his competence to work on the original project; committers before should remain committers afterward. Beyond that, you should express your desire to remain as compatible as possible with the fork, and say that you hope developers will port changes between the two whenever appropriate. If you have administrative access to the project's servers, publicly offer the forkers infrastructure help at startup time. For example, offer them a complete, deep-history copy of the version control repository, if there's no other way for them to get it, so that they don't have to start off without historical data (this may not be necessary depending on the version control system). Ask them if there's anything else they need, and provide it if you can. Bend over backward to show that you are not standing in the way, and that you want the fork to succeed or fail on its own merits and nothing else.

The reason to do all this—and do it publicly—is not to actually help the fork, but to persuade developers that your side is a safe bet, by appearing as non-vindictive as possible. In war it sometimes makes sense (strategic sense, if not human sense) to force people to choose sides, but in free software it almost never does. In fact, after a fork some developers often openly work on both projects, and do their best to keep the two compatible. These developers help keep the lines of communication open after the fork. They allow your project to benefit from interesting new features in the fork (yes, the fork may have things you want), and also increase the chances of a merger down the road.

Sometimes a fork becomes so successful that, even though it was regarded even by its own instigators as a fork at the outset, it becomes the version everybody prefers, and eventually supplants the original by popular demand. A famous instance of this was the GCC/EGCS fork. The *GNU Compiler Collection* (*GCC*, formerly the *GNU C Compiler*) is the most popular open source native-code compiler, and also one of the most portable compilers in the world. Due to disagreements between the GCC's official maintainers and Cygnus Software,[6] one of GCC's most active developer groups, Cygnus created a fork of GCC called *EGCS*. The fork was deliberately non-adversarial: the EGCS developers did not, at any point, try to portray their version of GCC as a new official version. Instead, they concentrated on making EGCS as good as possible, incorporating patches at a faster rate than the official GCC maintainers. EGCS gained in popularity, and eventually some major operating system distributors decided to package EGCS as their default compiler instead of GCC. At this point, it became clear to the GCC maintainers that holding on to the "GCC" name while everyone switched to the EGCS fork would burden everyone with a needless name change, yet do nothing to prevent the switchover. So GCC adopted the EGCS codebase, and there is once again a single GCC, but greatly improved because of the fork.

This example shows why you cannot always regard a fork as an unadulteratedly bad thing. A fork may be painful and unwelcome at the time, but you cannot necessarily know whether it will succeed. Therefore, you and the rest of the project should keep an eye on it, and be prepared not only to absorb features and code where possible, but in the most extreme case to even join the fork if it gains the bulk of the project's mindshare. Of course, you will often be able to predict a fork's likelihood of success by seeing who joins it. If the fork is started by the project's biggest complainer and joined by a handful of disgruntled developers who weren't behaving constructively anyway, they've essentially solved a problem for you by forking, and you probably don't need to worry about the fork taking momentum away from the original project. But if you see influential and respected developers supporting the fork, you should ask yourself why. Perhaps the project was being overly restrictive, and the best solution is to adopt into the mainline project some or all of the actions contemplated by the fork—in essence, to avoid the fork by becoming it.

Initiating a Fork

All the advice here assumes that you are forking as a last resort. Exhaust all other possibilities before starting a fork. Forking almost always means losing developers, with only an uncertain promise of gaining new ones later. It also means starting out with competition for users' attention: everyone who's about to download the software has to ask themselves: "Hmm, do I want that one or the other one?" Whichever one you are, the situation is messy, because a question has been introduced that wasn't there

[6] Now part of RedHat (http://www.redhat.com/).

before. Some people maintain that forks are healthy for the software ecosystem as a whole, by a standard natural selection argument: the fittest will survive, which means that, in the end, everyone gets better software. This may be true from the ecosystem's point of view, but it's not true from the point of view of any individual project. Most forks do not succeed, and most projects are not happy to be forked.

A corollary is that you should not use the threat of a fork as an extremist debating technique—"Do things my way or I'll fork the project!"—because everyone is aware that a fork that fails to attract developers away from the original project is unlikely to survive long. All observers—not just developers, but users and operating system packagers too—will make their own judgement about which side to choose. You should therefore appear extremely reluctant to fork, so that if you finally do it, you can credibly claim it was the only route left.

Do not neglect to take *all* factors into account in evaluating the potential success of your fork. For example, if many of the developers on a project have the same employer, then even if they are disgruntled and privately in favor of a fork, they are unlikely to say so out loud if they know that their employer is against it. Many free software programmers like to think that having a free license on the code means no one company can dominate development. It is true that the license is, in an ultimate sense, a guarantor of freedom—if others want badly enough to fork the project, and have the resources to do so, they can. But in practice, some projects' development teams are mostly funded by one entity, and there is no point pretending that that entity's support doesn't matter. If it is opposed to the fork, its developers are unlikely to take part, even if they secretly want to.

If you still conclude that you must fork, line up support privately first, then announce the fork in a non-hostile tone. Even if you are angry at, or disappointed with, the current maintainers, don't say that in the message. Just dispassionately state what led you to the decision to fork, and that you mean no ill will toward the project from which you're forking. Assuming that you do consider it a fork (as opposed to an emergency preservation of the original project), emphasize that you're forking the code and not the name, and choose a name that does not conflict with the project's name. You can use a name that contains or refers to the original name, as long as it does not open the door to identity confusion. Of course it's fine to explain prominently on the fork's home page that it descends from the original program, and even that it hopes to supplant it. Just don't make users' lives harder by forcing them to untangle an identity dispute.

Finally, you can get things started on the right foot by automatically granting all committers of the original project commit access to the fork, including even those who openly disagreed with the need for a fork. Even if they never use the access, your message is clear: there are disagreements here, but no enemies, and you welcome code contributions from any competent source.